

C++

Vojtěch Cimbura

Lecture III

11.11.2025

Quick Recap

- Last time:
 - Done TetrisBlock Actor: C++ and BP
 - WIP TetrisPlayer Pawn: C++ and BP
 - Member variables to hold Input assets
- TODO:
 - TetrisPlayer
 - Process player input
 - Notify GameMode about player input - sending message to GM via a Delegate
 - TetrisGameMode
 - Log player input from delegate call
 - Implement game logic - Game tick

TetrisPlayer - Processing Player Input (1/3)

- Setup the mapping context in `ATetrisPlayer::BeginPlay()`
 - Get player controller in order to access `ULocalPlayer`
 - Use that to get `UEnhancedInputLocalPlayerSubsystem*`
 - Add `TetrisMappingContext` as a mapping context to the obtained subsystem - Pawn will use our input mapping

```
// Called when the game starts or when spawned
void ATetrisPlayer::BeginPlay()
{
    Super::BeginPlay();

    check(TetrisMappingContext);

    if (APlayerController* PC = Cast<APlayerController>(Controller))
    {
        if (UEnhancedInputLocalPlayerSubsystem* Subsystem =
            ULocalPlayer::GetSubsystem<UEnhancedInputLocalPlayerSubsystem>(PC->GetLocalPlayer()))
        {
            Subsystem->AddMappingContext(TetrisMappingContext, 0);
        }
    }
}
```

TetrisPlayer - Processing Player Input (2/3)

- Create callback functions for player input in ATetrisPlayer

```
private:  
    void RotateBlockAction(const FInputActionValue& Value);
```

- Get current value of the action based on the type specified in IA_<Name>
 - Value.Get<TYPE>();
- Log a message

```
void ATetrisPlayer::RotateBlockAction(const FInputActionValue& Value)  
{  
    UE_LOG(LogTemp, Warning, TEXT("ROTATE"));  
}
```

- Do similarly for other input actions

TetrisPlayer - Processing Player Input (3/3)

- `ATetrisPlayer::SetupPlayerInputComponent()`
 - Get the Enhanced input component and bind our callback functions
 - Test that the message is being logged when the specified key is pressed

```
// Called to bind functionality to input
void ATetrisPlayer::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);

    check(RotateBlock);

    if (UEnhancedInputComponent* EIC = Cast<UEnhancedInputComponent>(PlayerInputComponent))
    {
        EIC->BindAction(RotateBlock, ETriggerEvent::Triggered, this,
            &ATetrisPlayer::RotateBlockAction);
    }
}
```

Forwarding input to GameMode

- Let's implement ability of **ATetrisPlayer** to call functions in **ATetrisGameMode** in C++
 - The simple way - direct reference

```
if (UWorld* World = GetWorld())
{
    if (ATetrisGameMode* GameMode = Cast<ATetrisGameMode>(World->GetAuthGameMode()))
    {
        GameMode->MoveBlockCalled(MoveDir);
    }
}
```

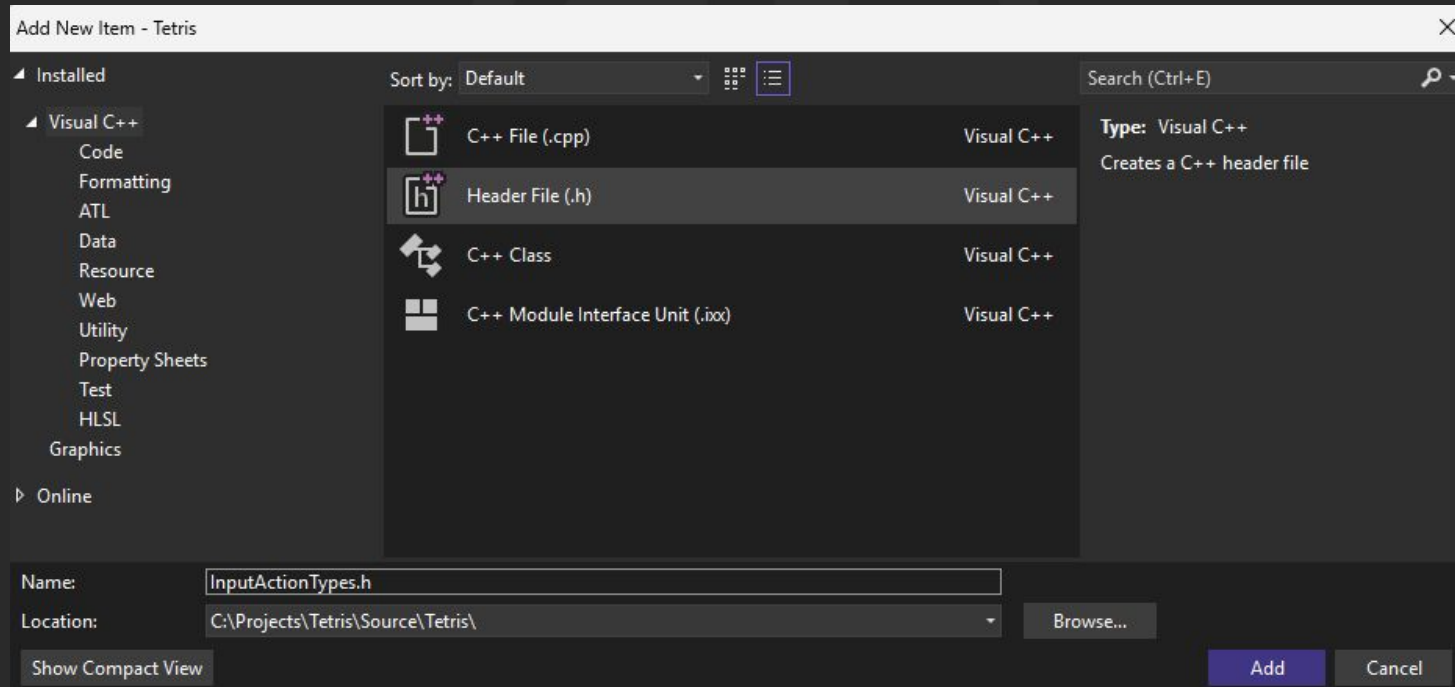
- Requires **ATetrisGameMode** function to be public
- If those are inaccessible objects (2 Actors) you need to connect them somehow so they know each other
 - Solution: **Delegates**

Delegates

- Data types that reference and execute member functions on C++ Objects
- Calling member functions on C++ objects in a generic, type-safe way
- Caller does not need to know on what object the function is being called
- **Single**, **Multicast**, and **Dynamic** delegates
 - Use Single delegate - only one object can be bound to it, we do not need it to be exposed to BP
- You can imagine that using them one can create complex messaging systems
- In our case, the delegate will carry a message specifying what action should be performed. For that, create a new enum class

InputActionTypes.h (1/2)

- Add a new file to the project, make sure it is created on the correct location



InputActionTypes.h (2/2)

- Specify 4 possible input actions that we will utilize
- Add one extra to define invalid value

```
#pragma once

#include "CoreMinimal.h"

UENUM()
enum class EInputActionType
{
    MoveBlockLeft,
    MoveBlockRight,
    PlaceBlock,
    RotateBlock,
    Undefined
};
```

Using a Delegate - Tetris Player

- Include "EInputActionTypes.h" in TetrisPlayer.h
- Below includes in TetrisPlayer.h, add the delegate definition

```
DECLARE_DELEGATE_OneParam(FOnGameInputRequested, EInputActionType);
```

- Add a public variable in TetrisPlayer

```
FOnGameInputRequested OnGameInputRequested;
```

- Add OnGameInputRequested.ExecuteIfBound() with correct enum parameter based on the action type in input callback functions

```
void ATetrisPlayer::RotateBlockAction(const FInputActionValue& Value)
{
    OnGameInputRequested.ExecuteIfBound(EInputActionType::RotateBlock);
}
```

Using a Delegate - Tetris GameMode (1/2)

- Include "EInputActionTypes.h" in TetrisGameMode.h
- Include "Kismet/GameplayStatics.h" in TetrisGameMode.cpp
- Create a callback function for the delegate with matching function parameter

```
void ProcessPlayerInput(EInputActionType InputType);
```

- In **BeginPlay()** function, bind the function to the Pawn delegate
- Based on the input type, log a message to the Output Log from the GameMode and verify in the Editor a correct message is logged.

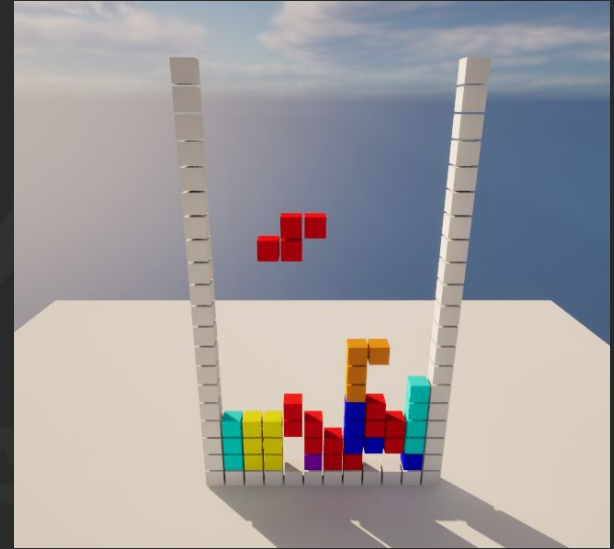
```
if (UWorld* World = GetWorld())
{
    if (ATetrisPlayer* PlayerPawn = Cast<ATetrisPlayer>(UGameplayStatics::GetPlayerPawn(World, 0)))
    {
        PlayerPawn->OnGameInputRequested.BindUObject(this, &ATetrisGameMode::ProcessPlayerInput);
    }
}
```

Using a Delegate - Tetris GameMode (2/2)

```
void ATetrisGameMode::ProcessPlayerInput(EInputActionType InputType)
{
    switch (InputType)
    {
        case EInputActionType::MoveBlockLeft:
            UE_LOG(LogTemp, Warning, TEXT("LEFT"));
            break;
        case EInputActionType::MoveBlockRight:
            UE_LOG(LogTemp, Warning, TEXT("RIGHT"));
            break;
        case EInputActionType::PlaceBlock:
            UE_LOG(LogTemp, Warning, TEXT("PLACE"));
            break;
        case EInputActionType::RotateBlock:
            UE_LOG(LogTemp, Warning, TEXT("ROTATE"));
            break;
        case EInputActionType::Undefined:
            UE_LOG(LogTemp, Warning, TEXT("<ERROR>"));
            break;
    }
}
```

The Game Logic

- “Only” the game logic is left at this point
- Steps:
 - Set up all Tetris pieces
 - Game Loop



Game logic idea

- Create a timer function that ticks in specified interval
 - Tick once in 50ms
 - Position of the falling piece will update once per 20 game ticks
 - Process player input once per game tick
- Have a falling piece stored in a struct, it will fall down / move through the board (store its position - X, Y - color, and rotation)
 - Update the board every tick
- Handle player's input:
 - Rotate block, move block down or L / R
- Detect when the piece cannot fall down anymore
 - Clear full lines if any, spawn a new piece if possible - game over otherwise
 - Update game score (Optional: increase difficulty)

Unreal Tick and Timers

- Tick
 - Engine distributes Tick events to subscribed objects that are fired in specified intervals
- Timers
 - Utilities used to schedule repeatable function invocations in specified intervals
 - UE has a timer manager, which takes timers, that are scheduled by the engine

UNREAL
ENGINE

Tetris GameMode - Timer

- Create a function called `ATetrisGameMode::TetrisGameTick()` and use it as the timer callback function

```
private:  
    void TetrisGameTick();
```

- Create a timer handle variable in `ATetrisGameMode`

```
UPROPERTY()  
Changed in 0 Blueprints  
FTimerHandle GameTickTimerHandle;
```

Tetris GameMode - Timer

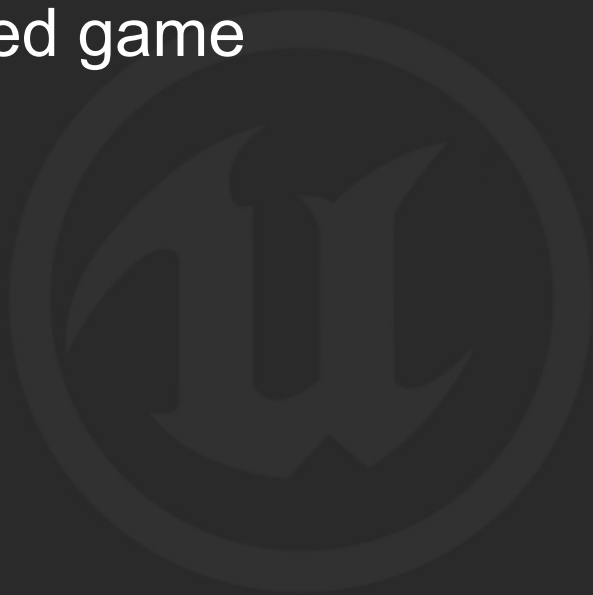
- Register a timer to the timer manager in `ATetrisGameMode::BeginPlay()`
 - Tick rate in seconds, let it tick each 50 ms
 - Store the handle to the timer so the timer can be modified later

```
// Tick each 50ms: 50ms / 1000ms => 20x / second
const float TickRate = 1.f / 20;
GetWorld()->GetTimerManager().SetTimer(
    GameTickTimerHandle, this, &ATetrisGameMode::TetrisGameTick, 1.f / 20.f, true);
```

- Log a simple message that will indicate that the timer is being invoked

```
void ATetrisGameMode::TetrisGameTick()
{
    UE_LOG(LogTemp, Warning, TEXT("Tick"));
}
```

Showcase of finished game



UNREAL
ENGINE

Internal Game Representation

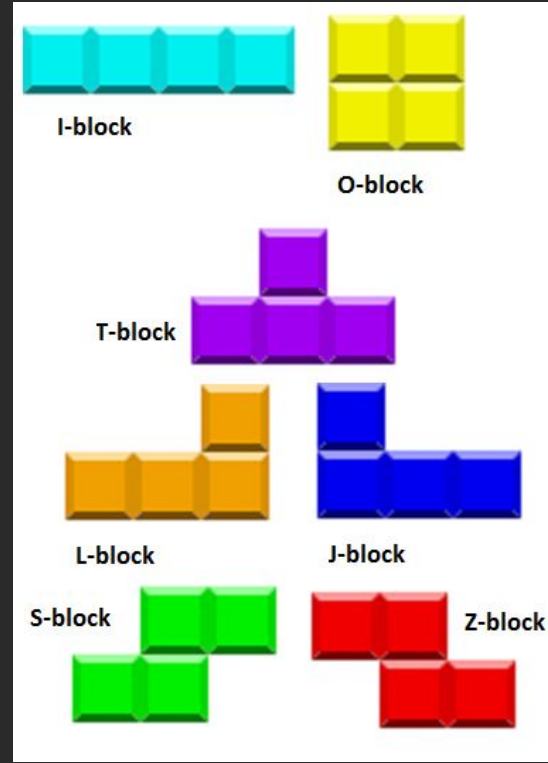
- Our array of `ATetrisBlock` actors we spawned is our “display” - each block is a “pixel”
- Create also an array of `uint8` variables of the same size where we will store the game board data
 - `TArray<uint8> InternalBoard;`
- Initialize it the same way as for the Board array (black or white)
- We'll store values 0-8:
 - 0 iff it is an empty space (**Black** Color) (Black color = hide actor)
 - 1 iff it is the border (**White** color)
 - ... next slide
- NOTE: Following logic assumes your board is spawned bottom→top. If it's vice-versa for you, some code needs to be changed

Tetris Pieces

```
struct FTetrisPiece
{
    TArray<uint8> Matrix;
};
```

- Create a struct to hold Tetris Pieces `FTetrisPiece`
 - Its member will be a `TArray` with 16x `uint8`
- Let's use 7 Tetris pieces:

○ I-Block		Idx = 2		Color = Cyan
○ O-Block		Idx = 3		Color = Yellow
○ T-Block		Idx = 4		Color = Purple
○ J-Block		Idx = 5		Color = Blue
○ L-Block		Idx = 6		Color = Orange
○ S-Block		Idx = 7		Color = Emerald
○ Z-Block		Idx = 8		Color = Red
- Store those structs in an array `TetrisPieces` while spawning in `ATetrisGameMode::BeginPlay()`



Tetris Pieces

- Similarly for remaining pieces

```
// I
FTetrisPiece Piece;
Piece.Matrix = TArray<uint8>
{
    0, 0, 0, 0,
    2, 2, 2, 2,
    0, 0, 0, 0,
    0, 0, 0, 0,
};
TetrisPieces.Push(Piece);

// O
Piece.Matrix = TArray<uint8>
{
    0, 0, 0, 0,
    0, 3, 3, 0,
    0, 3, 3, 0,
    0, 0, 0, 0,
};
TetrisPieces.Push(Piece);
```

Falling Piece

- New struct in `TetrisGameMode.h`
- Always contains data about the currently falling piece
 - Current board XY index
 - Current falling piece Tetris block color
 - Information about current rotation
- Define a new `ATetrisGameMode` variable `FFallingPiece FallingPiece`

```
struct FFallingPiece
{
    int32 PositionX = 0;
    int32 PositionY = 0;
    uint8 CurrentColor = 0;
    uint8 CurrentRotation = 0;
};
```

```
/** Data representation of the falling piece */
FFallingPiece FallingPiece;
```

Falling Piece

- Create function `ATetrisGameMode::SetupNewPiece()`, where you will initialize a random tetris piece on the board
 - Color: range 2-8
 - Rotation = 2 (180° rotated as board indexed bottom->top, pieces top->bottom)
- Call it at the end of `ATetrisGameMode::BeginPlay()`

```
void ATetrisGameMode::SetupNewPiece()
{
    FallingPiece.CurrentColor = FMath::RandRange(0, TetrisPieces.Num() - 1) + 2;    // Random tetris piece (2-8 are legal colors)
    FallingPiece.CurrentRotation = 2;    // 180 deg because pieces are in the matrix top->bottom and board is bottom->top
    FallingPiece.PositionX = BoardWidth / 2 - 2;    // In the middle if possible (a half, without border blocks)
    FallingPiece.PositionY = BoardHeight - 4;    // Offset the top by tetris piece height (4x4)
```

Game Loop Functions in `ATetrisGameMode`

- `DecodeBlockColor()`
 - Translate `InternalBoard` representation (`uint8`) into `FColor`
- `GetIndexBasedOnRotation()`
 - Compute local index in the 4x4 tetris piece while the piece rotates
- `CanPlacePiece()`
 - True when the falling piece can be placed on given location, false otherwise
- `UpdateBoard()`
 - Copy internal board state into the “Display” - set block color
- `PlacePiece()`
 - Incorporate the `FallingPiece` into the board representation

FColor DecodeBlockColor(uint8 BlockColorType)

```
FColor RetColor = FColor::White;

switch (TetrisBlockType)
{
case 0: RetColor = FColor::Black; break;
case 1: RetColor = FColor::White; break;
case 2: RetColor = FColor::Cyan; break;
case 3: RetColor = FColor::Yellow; break;
case 4: RetColor = FColor::Purple; break;
case 5: RetColor = FColor::Blue; break;
case 6: RetColor = FColor::Orange; break;
case 7: RetColor = FColor::Emerald; break;
case 8: RetColor = FColor::Red; break;
}

return RetColor;
```

uint8 GetIndexBasedOnRotation(PosX, PosY, Rot)

- Support only CW rotation

- Rot = 0 (0 deg) | $\text{Idx} = (4 * \text{PosY}) + \text{PosX};$
- Rot = 1 (90 deg) | $\text{Idx} = 12 + \text{PosY} - (4 * \text{PosX});$
- Rot = 2 (180 deg) | $\text{Idx} = 15 - (4 * \text{PosY}) - \text{PosX};$
- Rot = 3 (270 deg) | $\text{Idx} = 3 - \text{PosY} + (4 * \text{PosX});$

0	1	2	3		12	8	4	0		15	14	13	12		3	7	11	15
4	5	6	7		13	9	5	1		11	10	9	8		2	5	8	11
8	9	10	11		14	10	6	2		7	6	5	4		1	4	7	10
12	13	14	15		15	11	7	3		3	2	1	0		0	3	6	9

uint8 GetIndexBasedOnRotation(PosX, PosY, Rot)

```
uint8 LocalIndex = 0;

switch (CurrentRotation)
{
case 0: LocalIndex = PosY * 4 + PosX; break;
case 1: LocalIndex = 12 + PosY - (PosX * 4); break;
case 2: LocalIndex = 15 - (PosY * 4) - PosX; break;
case 3: LocalIndex = 3 - PosY + (PosX * 4); break;
}

return LocalIndex;
```

uint8 CanPlacePiece(PosX, PosY, Rotation)

- For each falling piece idx, get the correct index based on rotation
 - Utilize `GetIndexBasedOnRotation()`
- Get correct board XY coordinates of the falling piece block
- Compute `BoardIndex` - mapping into 1D array
- If within board bounds, check if:
 - Current Piece we are trying to place has got a non-zero value on `LocalPieceIndex` in Matrix and `InternalBoard` has non-zero value on `BoardIndex`
 - If yes, Piece cannot be placed - return 0
- If all checks pass, return 1
- **Note:** We return 1 even if a part of the piece's matrix does not fit the board! This is OK as long as we care about the occupied block pieces and not empty spaces in the matrix

uint8 CanPlacePiece(PosX, PosY, Rotation)

```
// For each block in the piece matrix try to find out if it is within board bounds
// We return 1 even if a part of the piece's matrix does not fit the board!
// This is OK as long as we care about the occupied block pieces and not empty spaces in the matrix
for (int32 PosY = 0; PosY < 4; ++PosY)
{
    for (int32 PosX = 0; PosX < 4; ++PosX)
    {
        uint8 LocalPieceIndex = GetIndexBasedOnRotation(PosX, PosY, PieceRotation);

        int32 BoardCoordsX = PieceX + PosX;
        int32 BoardCoordsY = PieceY + PosY;

        int32 BoardIndex = BoardCoordsY * BoardWidth + BoardCoordsX;

        // Check if we are within bounds
        if (BoardCoordsX > 0 ||
            BoardCoordsX < BoardWidth - 1 ||
            BoardCoordsY > 0 ||
            BoardCoordsY < BoardHeight - 1
        )
        {
            // We are within board bounds for this piece's block
            // Now check if the tested piece block is occupied and the board is occupied
            // if yes, then the Falling Piece cannot be placed at this location
            if (TetrisPieces[FallingPiece.CurrentColor - 2].Matrix[LocalPieceIndex] != 0 && InternalBoard[BoardIndex] != 0)
            {
                return 0;
            }
        }
    }
}
return 1;
```

void UpdateBoard()

- The `FallingPiece` is NOT incorporated into the `InternalBoard` yet!
- Two steps to update the board at each tick:
 - 1) Copy the `InternalBoard` representation to the 'Display'
 - 2) Update the `FallingPiece` position
 - Go through the 4x4 Matrix of `FallingPiece` while getting the local piece index at current `FallingPiece` rotation
 - If the current block is non-zero, compute `BoardIndex` using Board coordinates of the `FallingPiece`
 - If within the board bounds, update the color of the block on given `BoardIndex` using `ATetrisBlock::SetBlockColor()` to the falling piece color
- Call `UpdateBoard()` function at the end of `TetrisGameTick()` function
 - You should be able to see the spawned piece when hitting Play

void UpdateBoard() (1/2)

- Copy the `InternalBoard` representation to the 'Display'

```
// Update the board state
for (uint8 y = 0; y < BoardHeight; ++y)
{
    for (uint8 x = 0; x < BoardWidth; ++x)
    {
        uint8 CurrentIdx = y * BoardWidth + x;

        ATetrisBlock* BoardBlock = Board[CurrentIdx];
        FColor BoardBlockColor = BoardBlock->GetBlockColor();
        FColor CurrentBlockColor = DecodeBlockColor(InternalBoard[CurrentIdx]);

        if (BoardBlockColor != CurrentBlockColor)
        {
            BoardBlock->SetBlockColor(CurrentBlockColor);
        }
    }
}
```

void UpdateBoard() (2/2)

- Update the **FallingPiece** position

```
// Update the falling piece
for (int32 PosY = 0; PosY < 4; ++PosY)
{
    for (int32 PosX = 0; PosX < 4; ++PosX)
    {
        uint8 LocalPieceIndex = GetIndexBasedOnRotation(PosX, PosY, FallingPiece.CurrentRotation);

        if (TetrisPieces[FallingPiece.CurrentColor - 2].Matrix[LocalPieceIndex] != 0)
        {
            // Current index is a part of the tetris piece
            int32 BoardCoordsX = FallingPiece.PositionX + PosX;
            int32 BoardCoordsY = FallingPiece.PositionY + PosY;

            int32 BoardIndex = BoardCoordsY * BoardWidth + BoardCoordsX;

            // Check if it is within the board limits
            if (BoardCoordsX > 0 ||
                BoardCoordsX < BoardWidth - 1 ||
                BoardCoordsY > 0 ||
                BoardCoordsY < BoardHeight - 1
            )
            {
                Board[BoardIndex]->SetBlockColor(DecodeBlockColor(FallingPiece.CurrentColor));
            }
        }
    }
}
```

void PlacePiece()

- Almost identical code as for the `UpdateBoard()` step 2 except the action to perform:
 - Go through the 4x4 Matrix of `FallingPiece` while getting the local piece index at current `FallingPiece` rotation
 - If the current block is non-zero, compute `BoardIndex` using Board coordinates of the `FallingPiece`
 - If within the board bounds, set the `InternalBoard` value at `BoardIndex` to the color in the current `FallingPiece`

UNREAL
ENGINE

void PlacePiece()

```
void ATetrisGameMode::PlacePiece()
{
    // For each piece coordinate try to place it into the internal board
    for (int32 PosY = 0; PosY < 4; ++PosY)
    {
        for (int32 PosX = 0; PosX < 4; ++PosX)
        {
            uint8 LocalPieceIndex = GetIndexBasedOnRotation(PosX, PosY, FallingPiece.CurrentRotation);

            if (TetrisPieces[FallingPiece.CurrentColor - 2].Matrix[LocalPieceIndex] != 0)
            {
                // Current index is a part of the tetris piece
                int32 BoardCoordsX = FallingPiece.PositionX + PosX;
                int32 BoardCoordsY = FallingPiece.PositionY + PosY;

                int32 BoardIndex = BoardCoordsY * BoardWidth + BoardCoordsX;

                // Check if it is within the board limits
                if (BoardCoordsX > 0 ||
                    BoardCoordsX < BoardWidth - 1 ||
                    BoardCoordsY > 0 ||
                    BoardCoordsY < BoardHeight - 1
                )
                {
                    InternalBoard[BoardIndex] = FallingPiece.CurrentColor;
                }
            }
        }
    }
}
```

TetrisGameMode - Player's input processing

- Let's incorporate the user input at the start of `TetrisGameTick()`
 - New vars: `bMoveLeft`, `bMoveRight`, `bRotate`, `bPlace`
 - If `bMoveLeft` is true, lower `FallingPiece.PositionX` by 1 if the Piece can be placed on position `FallingPiece.PositionX - 1`
 - Reset `bMoveLeft` to false
 - Similarly for `bMoveRight`, just `FallingPiece.PositionX + 1`
 - When `bPlace` is true, lower `FallingPiece.PositionY - 1` if the Piece can be placed there , but DO NOT reset `bPlace` to false!
 - This is because `IA_PlaceBlock` is called on Press and Release
 - For `bRotate`, try to place the block on the same position but the next rotation is $(CurrRot + 1) \% 4$
 - Reset `bRotate` to false

TetrisGameMode - Player's input processing (1/2)

- New variables: `bMoveLeft`, `bMoveRight`, `bRotate`, `bPlace`
 - Either make them bools or you can use bitfield initialization

```
uint8 bMoveLeft : 1;  
uint8 bMoveRight : 1;  
uint8 bMoveDown : 1;  
uint8 bRotate : 1;
```

UNREAL
ENGINE

TetrisGameMode - Player's input processing (2/2)

```
void ATetrisGameMode::TetrisGameTick()
{
    // Process player's input
    if (bMoveLeft)
    {
        FallingPiece.PositionX -= CanPlacePiece(FallingPiece.PositionX - 1, FallingPiece.PositionY, FallingPiece.CurrentRotation);
        bMoveLeft = false;
    }
    if (bMoveRight)
    {
        FallingPiece.PositionX += CanPlacePiece(FallingPiece.PositionX + 1, FallingPiece.PositionY, FallingPiece.CurrentRotation);
        bMoveRight = false;
    }
    if (bMoveDown)
    {
        FallingPiece.PositionY -= CanPlacePiece(FallingPiece.PositionX, FallingPiece.PositionY - 1, FallingPiece.CurrentRotation);
    }
    if (bRotate)
    {
        FallingPiece.CurrentRotation += CanPlacePiece(FallingPiece.PositionX, FallingPiece.PositionY, (FallingPiece.CurrentRotation + 1) % 4);
        FallingPiece.CurrentRotation = FallingPiece.CurrentRotation % 4;
        bRotate = false;
    }
}
```

Game Tick Implementation

- Create variables in `ATetrisGameMode`:
 - `uint32 CurrentTick`
 - `uint32 MoveBlockDownTicks`
 - = How many timer ticks to process before moving the falling piece one block down
 - `uint32 Score`
 - `uint32 PlacedBlocks`
- Each `TetrisGameTick()` function invocation, increase the variable `CurrentTick`
- If it reaches `MoveBlockDownTicks` value:
 - Reset `CurrentTick` to 0

Game Tick Implementation

```
// Process this tick
CurrentTick++;

if (CurrentTick == MoveBlockDownTicks)
{
    CurrentTick = 0;

    // TODO
}

// Update board
UpdateBoard();
```

Game Tick Implementation

- Check if you can place the falling piece on position one below (`CanPlacePiece()`)
 - If yes, decrement the `FallingPiece.PositionY` variable
 - If not, the piece cannot move - place it
 - Call `PlacePiece()`
 - Create function `UpdateScore()` where you add a value to the score value and increment `PlacedBlocks` variable
 - Each 15 placed blocks decrement `MoveBlockDownTicks` variable - makes the game harder
 - Call `SetupNewPiece()`
 - **Return!**
 - Otherwise the new piece will be visualized in this tick even if technically we just set it up in this tick!

Game Tick Implementation

```
if (CurrentTick == MoveBlockDownTicks)
{
    CurrentTick = 0;

    if (CanPlacePiece(FallingPiece.PositionX, FallingPiece.PositionY - 1, FallingPiece.CurrentRotation))
    {
        // Yep, it can fall down
        --FallingPiece.PositionY;
    }
    else
    {
        // Falling piece reach a position where it cannot move down anymore
        // Fix it to the board, check for lines, update score and spawn a new piece
        PlacePiece();

        UpdateScore();

        SetupNewPiece();

        // Dont update the board here and exit this tick immediately
        // otherwise the new piece will be visualized in this tick even if technically
        // we just set it up in this tick
        return;
    }
}
```

Game Tick Implementation

- Example implementation of `UpdateScore()` function

```
void ATetrisGameMode::UpdateScore()
{
    PlacedBlocks++;

    Score += 10;

    if (PlacedBlocks % 15 == 0)
    {
        // Increase tick speed of the falling block
        --MoveBlockDownTicks;
        UE_LOG(LogTemp, Warning, TEXT("GETTING FASTER!"));
    }
}
```

ENGINE

Game Tick Implementation

- Game should run at this point, the block falls down until it cannot move anymore and a new piece is spawned
- Final steps:
 - Clearing filled lines & updating the score
 - Detecting game over

UNREAL
ENGINE

void CheckForLineFills() (1/6)

- Create variable in `ATetrisGameMode`:
 - `TArray<uint8> FilledLines`
- Create function `ATetrisGameMode::CheckForLineFill()`
- (Optional) If there are any elements in the `FilledLines` array, add more score points in the function `UpdateScore()`

UNREAL
ENGINE

void CheckForLineFills() (2/6)

- First part:
 - Scan the board bottom to top, from left to right, ignore the border blocks
 - If there isn't a block on the current position, continue with the line above
 - If you scanned the whole line and there were blocks, push the current Y variable in the **FilledLines** array to be used later
- After this part, the **FilledLines** array contains Y board coordinates that are fully covered with blocks

UNREAL
ENGINE

void CheckForLineFills() (3/6)

```
void ATetrisGameMode::CheckForLineFill()
{
    // Scan the board bottom to top...
    for (int32 PosY = 1; PosY < BoardHeight - 1; ++PosY)
    {
        bool bLine = true;

        // ... from left to right, without border blocks ....
        for (int32 PosX = 1; PosX < BoardWidth - 1; ++PosX)
        {
            // ... and if there isn't block on the current position, immediately go for the line above
            if (InternalBoard[PosY * BoardWidth + PosX] == 0)
            {
                bLine = false;
                break;
            }
        }

        // If we got here with bLine = true, then the line was full of blocks
        if (bLine)
        {
            // ... so store the current height (Y line coordinate) where the board is full
            FilledLines.Push(PosY);
        }
    }
}
```

void CheckForLineFills() (4/6)

- Second part:
 - If any lines should be cleared (`FilledLines` is not empty), go through all of them
 - For each entry, start on the X axis and iterate up while moving each block down by one
 - After you clear the line and blocks are moved, decrease all values in the `FilledLines` array if there are any elements left
 - We moved all above blocks down, so `FilledLines` array values are outdated - we need to decrease them all by 1
 - Clear the `FilledLines` array
 - You can postpone this action until you update the score in `UpdateScore()`, but don't forget to do it eventually

void CheckForLineFills() (5/6)

```
// If any lines should be cleared ...
if (FilledLines.Num() > 0)
{
    // ... go through them
    for (uint8 i = 0; i < FilledLines.Num(); ++i)
    {
        uint8 LineY = FilledLines[i];

        // Now start on the x axis, ...
        for (int32 PosX = 1; PosX < BoardWidth - 1; ++PosX)
        {
            // ... iterate up ...
            for (int32 PosY = LineY; PosY < BoardHeight - 1; ++PosY)
            {
                // ... and move each block down
                InternalBoard[PosY * BoardWidth + PosX] = InternalBoard[(PosY + 1) * BoardWidth + PosX];
            }
        }

        // Line was now cleared. But if there were any more to be cleared...
        if (i < FilledLines.Num() - 1)
        {
            // ... we need to go through them ...
            for (uint8 j = i; j < FilledLines.Num(); ++j)
            {
                // ... and lower their Y coordinate, because those were pushed one down
                --FilledLines[j];
            }
        }
    }
}
```

void CheckForLineFills() (6/6)

- Call it when placing the falling piece

```
1 // Falling piece reach a position where it cannot move down anymore  
2 // Fix it to the board, check for lines, update score and spawn a new piece  
3 PlacePiece();  
  
4 CheckForLineFill();  
  
5 UpdateScore();  
  
6 SetupNewPiece();  
  
7 // Dont update the board here and exit this tick immediately  
8 // otherwise the new piece will be visualized in this tick even if technically  
9 // we just set it up in this tick  
10 return;
```

Detecting Game Over

- If the new piece cannot be placed at the default location in `SetupNewPiece()`:
 - Clear and Invalidate the `GameTickTimerHandle`
 - Log the current score
- We have a working Tetris clone :-)

```
void ATetrisGameMode::SetupNewPiece()
{
    FallingPiece.CurrentColor = FMath::RandRange(0, TetrisPieces.Num() - 1) + 2;    // Random tetris pi
    FallingPiece.CurrentRotation = 2;    // 180 deg because pieces are in the matrix top
    FallingPiece.PositionX = BoardWidth / 2 - 2;    // In the middle if possible (a half, without b
    FallingPiece.PositionY = BoardHeight - 4;    // Offset the top by tetris piece height (4x4)

    if (!CanPlacePiece(FallingPiece.PositionX, FallingPiece.PositionY, FallingPiece.CurrentRotation))
    {
        // Game over, cannot place a new piece
        GetWorldTimerManager().ClearTimer(GameTickTimerHandle);
        GameTickTimerHandle.Invalidate();
        UE_LOG(LogTemp, Error, TEXT("GameOver! Score: %d"), Score);
    }
}
```

Learning material

- [UE Naming convention](#)
- [Programming in UE C++, the whole documentation](#)
- [Balancing BPs and C++](#)
- [Getting started with Unreal Engine](#)
- [Asserts in UE](#)
- [Objects in UE](#)
- [Unreal Engine Reflection System](#)
- [Advanced Delegates](#)

UNREAL
ENGINE